

FRONT-END

DEVELOPER

2026.03

PORTFOLIO

김예찬 KIM YECHAN

010 - 6383 - 8295

az20058@naver.com

<https://web-chan.tistory.com/>

PROJECT

01

(주)법틀 - BuptleBiz 2025. 4 ~

기업 법무팀의 빠른 업무 속도 및 안정성을 제공하기 위해 AI를 사용해 계약, 법무 관리 등의 기능을 제공하는 웹 솔루션

02

KORAD - 경화주월 2024. 8 ~ 2024. 10

잘 알려지지 않은 원자력환경공단을 경주 관광지로서 홍보하고, 공단의 관리 안정성 또한 나타내기 위해 제작한 웹앱 프로젝트

03

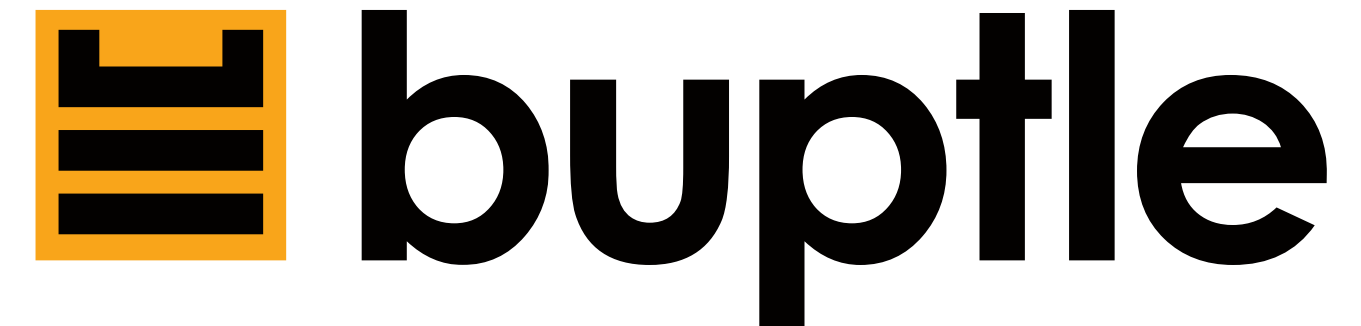
SKUMARU - 학교 사이트 재구축 2024. 10 ~ 2026. 2

대학교 생활에 있어서 흩어지고 복잡한 정보들을 하나로 모으고, 학우들의 불편함을 개선하기 위해 제작한 프로젝트

PROJECT 01.

(주)법틀 - BuptleBiz

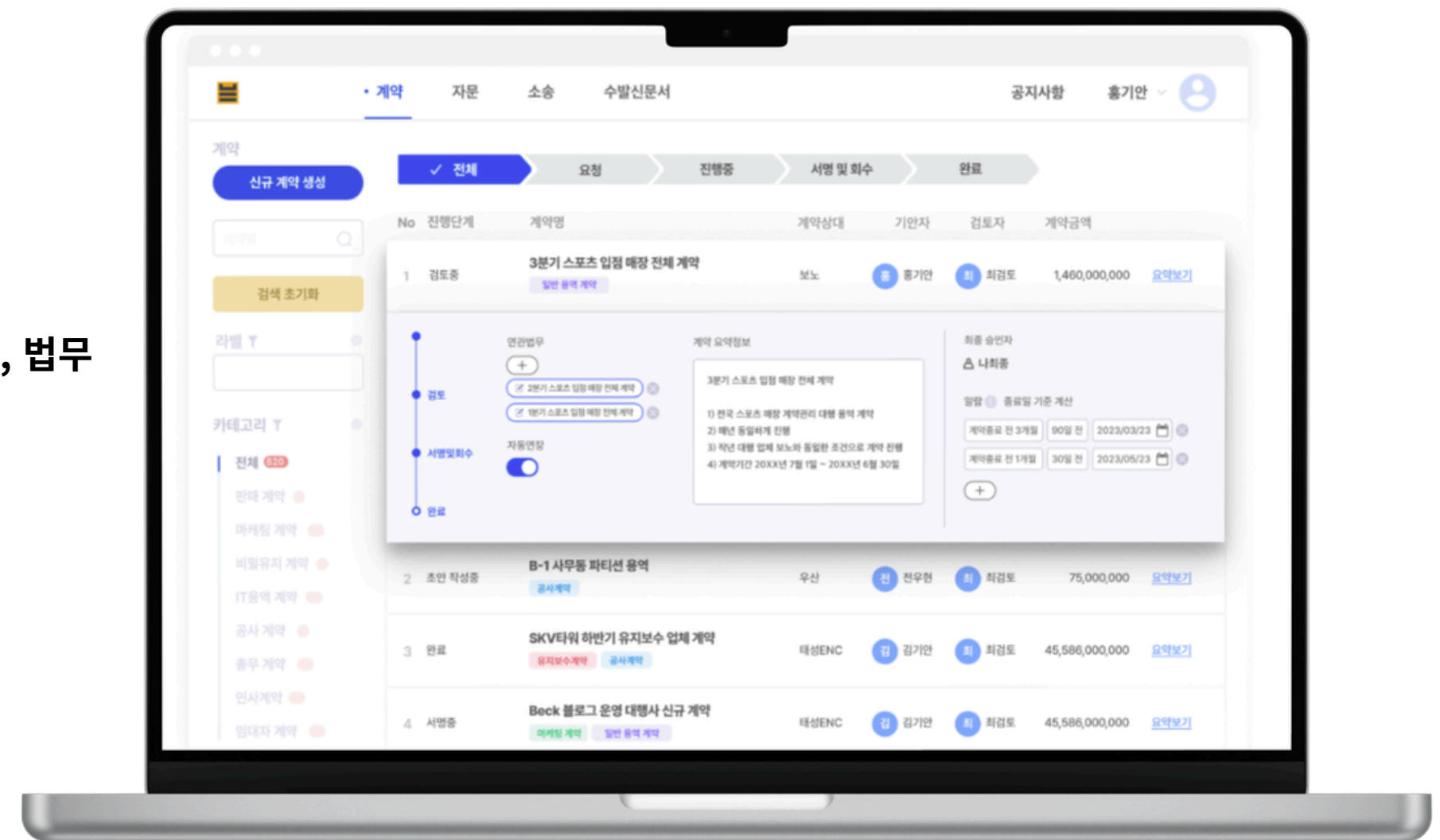
2025. 4 ~ (재직 중)



프로젝트 개요

기업 법무팀의 빠른 업무 속도 및 안정성을 제공하기 위해 AI를 사용해 계약, 법무 관리 등의 기능을 제공하는 웹 솔루션

React, Next, Typescript, React-Query, Zustand, Tailwind, Django, python, docker, AWS EC2, Recharts, Context-Api, Vitest, MSW



트러블 슈팅 | 1. 배포 지연 개선을 위한 단위 테스트 도입 [프로젝트 회고 링크](#)

문제 상황

고객사 요구사항이 증가하면서 통계 페이지 복잡도가 지속적으로 증가했습니다. 이에 따라 코드 수정 시에 사이드 이펙트 발생 확률이 증가하여 안정성은 감소했고, 평균 배포 지연 시간은 1시간 이상이었습니다. 버그 식별 시간을 단축하고, 배포 지연을 단축하기 위해 통계 페이지에 테스트 코드를 도입했습니다.

해결 과정

팀의 목표는 '공통 함수 수정에 따른 사이드 이펙트 방지'였고, 회사 프로젝트와 테스트 코드 작성을 병행해야 했기 때문에 최소한의 리소스로 테스트 코드를 작성해야 했으므로 단위 테스트 적용을 결정했습니다. 테스트 케이스는 우선순위가 높은 순인 P1, P2, P3로 나누어 P1, P2 케이스 위주로 작성했습니다. P1은 오류 발생 시 장애 등급 S를 발생시키며 수정 확률이 높고, P3는 다른 테스트에 의해 간접적으로 커버되며 수정 확률이 낮습니다. AI agent를 활용하여 중요 로직에서 사용되는 함수, 테스트가 중복되는 함수 등을 쉽게 찾아냈고, 각 조건을 중심으로 테스트 우선순위를 분류했습니다. 위와 같은 기준으로 약 200개의 통계 관련 테스트 케이스를 작성했고, 평균 코드 커버리지 82%를 확보했습니다. 이를 통해 배포 지연과 개발 생산성을 개선할 수 있었습니다.

결과 및 배운 점

테스트 코드 도입을 통해 평균 배포 지연 시간은 약 50% 이상 감소했고, 그중 사이드 이펙트에 의한 배포 지연은 거의 0%에 수렴하게 되었습니다. 테스트 코드 작성을 하면 꽤 많은 리소스가 필요하므로, 무조건 높은 커버리지를 확보하는 것보다 팀의 목표와 상황에 따라 적절한 코드 커버리지를 해야 한다는 것을 배웠습니다.

File #s	% Stmts	% Branch	% Funcs	% Lines	Uncovered Line
src/pages/statistics	85.29	82.35	88.46	85.29	
StatisticsPage.tsx	91.30	85.71	100	91.30	45,78
StatisticsChart.tsx	82.60	82.35	83.33	82.60	54-57,72
StatisticsFilter.tsx	88.88	83.33	85.71	88.88	28

테스트 커버리지 결과 출력(일부)

P1 Case

- 오류 발생 시 장애 등급 S~A 발생
- 수정 확률 높음
- 코드 복잡도 높음

P2 Case

- 오류 발생 시 장애 등급 C~B 발생
- 수정 확률 비교적 낮음

P3 Case

- 다른 테스트에 의해 간접적으로 커버됨
- 매우 간단한 로직

테스트 케이스 우선순위 결정 과정

트러블 슈팅 | 2. 프론트엔드 개발 환경 개선 [🔗 프로젝트 회고 링크](#)

문제 상황

자사 프론트엔드 환경은 React + CRA 로 구성되어 있었습니다. 프로젝트 규모가 커짐에 따라 개발 서버 성능이 저하되었고, 이는 종종 일정 병목을 일으켰습니다. 개발 서버 시작은 평균 30초가 소요되었고, HMR은 평균 5~10초가 소요되었습니다. 이 문제를 해결하기 위해 우리 팀은 2개의 대안을 검토하게 되었습니다.

해결 과정

두 방법은 기존 'Webpack 최적화'와 'Vite 마이그레이션'이었습니다. 우리 팀은 다음과 같이 트레이드오프를 분석했습니다. Vite는 마이그레이션 및 새 번들러 학습 비용으로 인해 초기 리소스가 크지만, ESM Native 동작 방식으로 인해 번들링 과정이 제외되므로 프로젝트 규모에 관계 없이 비교적 빠른 개발 속도를 지원합니다. Webpack은 초기 비용이 작지만, CommonJS 방식을 사용하여 모든 코드 베이스를 번들링하므로 프로젝트 규모가 크다면, 그만큼 성능이 저하됩니다. 실제 테스트 결과, Vite가 최적화된 Webpack에 비해 평균 30% 이상 개발 서버 속도가 빨랐습니다. 또한 레거시가 된 Webpack을 사용했을 때 기술 부채 발생과 신규 팀원의 적응 시간도 고려해야 했습니다. 우리 팀은 '개발 서버 속도'와 '추후 관리 비용'을 판단 근거로 하여 Vite 마이그레이션을 결정했습니다. 또한 빌드 스크립트를 Vite의 rollup 기반 설정 파일로 재작성하고, 빌드 속도 개선을 위해 코드 스플리팅 및 모듈에 따른 청크 분할도 추가했습니다.

결과 및 배운 점

결과적으로 기존 Webpack 환경에 비해 Vite에서 오른쪽 차트와 같이 개선되었고, 그 과정에서 코드 스플리팅(동적 import)도 진행했기 때문에 번들 사이즈와 초기 로딩 또한 개선할 수 있었습니다.

Webpack

CommonJS

코드 변경 → 청크 재번들링 → 브라우저

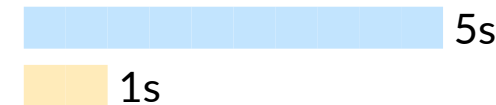
Vite

Native ESM

코드 변경 → 해당 모듈만 변경 → 브라우저

Webpack, Vite HMR 방식

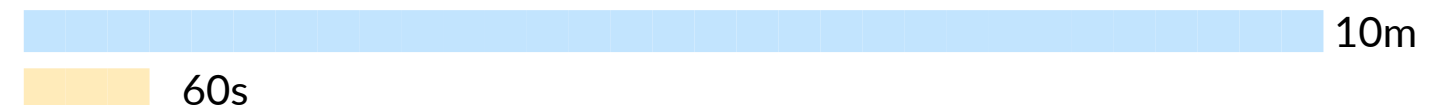
HMR 소요 시간



개발 서버 시작 소요 시간



빌드 소요 시간



Webpack → Vite 성능 개선

트러블 슈팅 | ISR 적용으로 응답 속도 및 서버 부하 개선

문제 상황

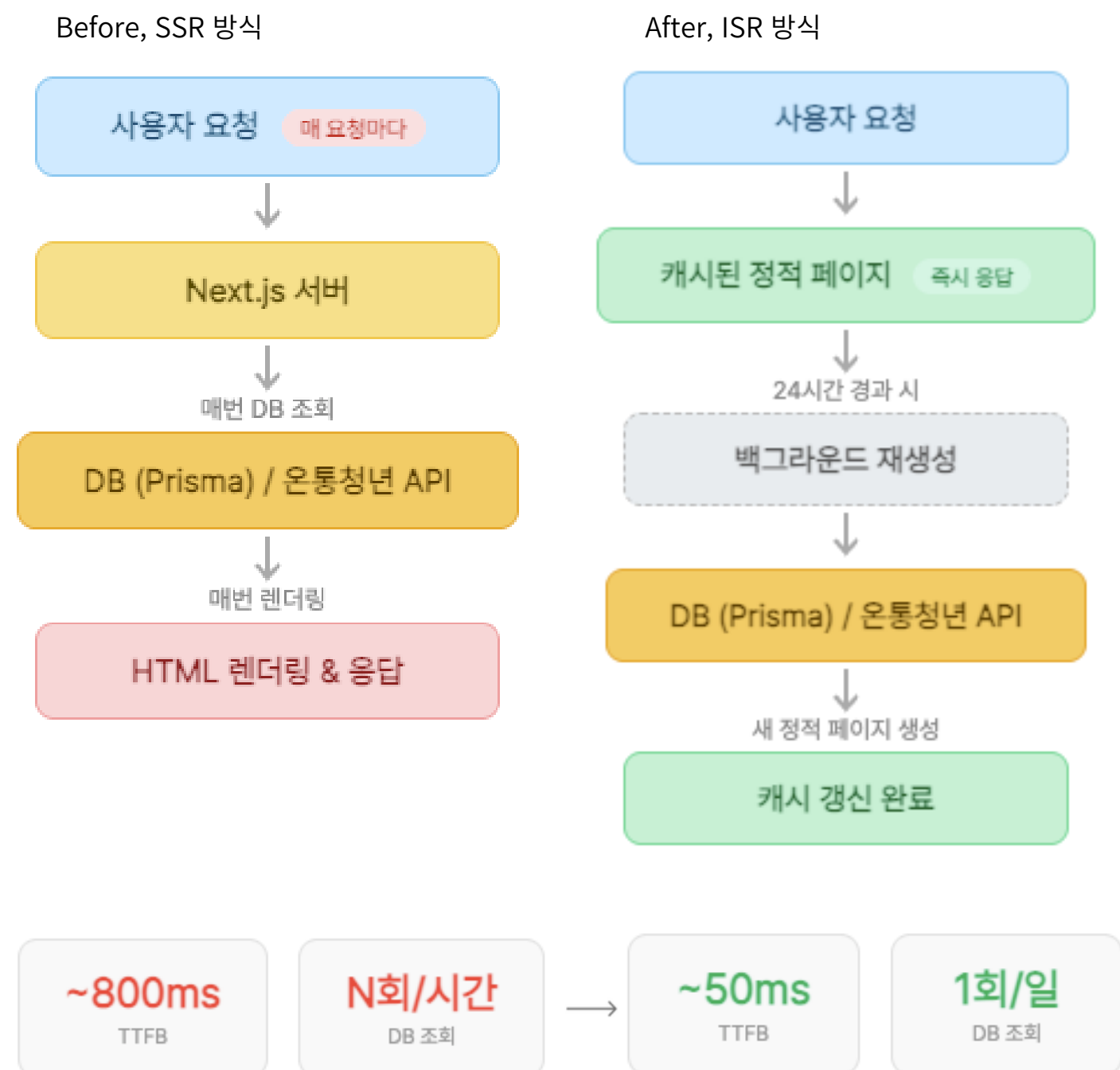
정책 둘러보기 페이지는 매 요청마다 DB를 조회하는 SSR 방식으로 구현되어 있었습니다. 그러나 정책 데이터는 하루 1회 배치 Job으로 갱신되는 특성상 실시간 조회가 불필요했고, 사용자 유입이 가장 많은 페이지임에도 매번 DB를 호출하여 불필요한 서버 부하와 느린 응답 속도를 초래했습니다. 이를 해결하기 위해 SSG와 ISR을 검토하게 되었습니다.

해결 과정

SSG는 빌드 시점에 정적 페이지를 생성하여 가장 빠른 응답 속도를 보장하지만, 정책 데이터가 갱신될 때마다 전체 재빌드가 필요하여 배치 Job과의 연동이 복잡해지는 단점이 있었습니다. ISR은 정적 페이지를 생성하되, 설정한 주기마다 백그라운드에서 페이지를 재생성하여 최신 데이터를 반영할 수 있어 SSG의 성능과 SSR의 데이터 신선도를 동시에 확보할 수 있었습니다. 판단 기준은 '데이터의 갱신 주기와 페이지의 응답 성능을 어떻게 균형 있게 맞출 수 있는가'였습니다. 정책 데이터는 하루 1회 갱신되므로 실시간 반영이 불필요했고, SSG처럼 재빌드 파이프라인을 구축하는 비용 대비 ISR의 revalidate 설정 한 줄로 동일한 효과를 얻을 수 있었기 때문에 ISR을 채택했습니다. 정책 목록 페이지에 86400의 revalidate를 설정하여 1일 주기로 페이지를 재생성하도록 했고, API 라우트에도 Cache-Control 헤더를 적용하여 클라이언트 측 캐싱까지 확보했습니다.

결과 및 배운 점

ISR 적용 후 정책 목록 페이지의 TTFB가 평균 800ms에서 50ms 이하로 단축되었고, DB 조회 횟수가 시간당 수백 회에서 1회로 감소하여 서버 부하를 대폭 줄일 수 있었습니다. 또한, 데이터의 갱신 주기에 맞는 렌더링 전략을 선택하는 것이 성능 최적화의 핵심이라는 것을 배웠습니다.



PROJECT 03.

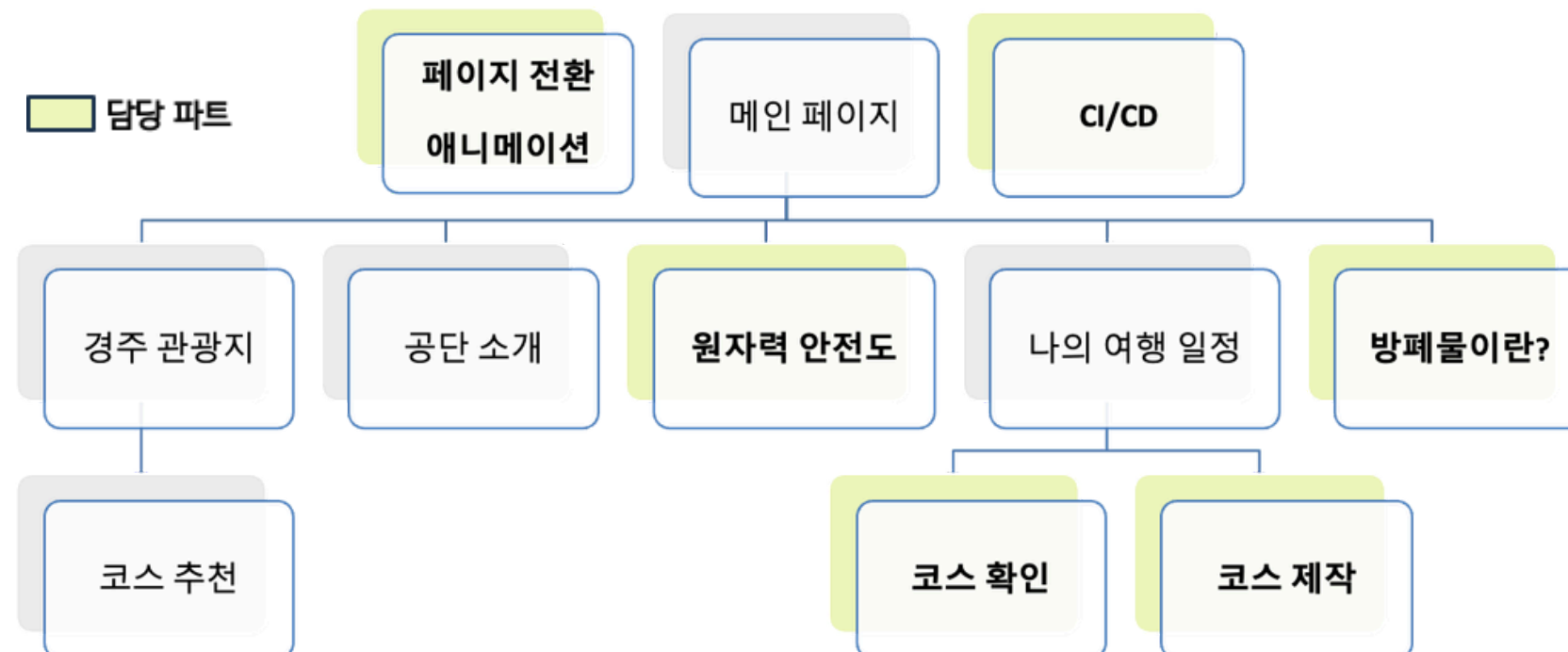
KORAD - 경화주월

2024. 8 ~ 2024. 10

프로젝트 개요

잘 알려지지 않은 **원자력환경공단**을 경주 관광지로서 **홍보**하고, 공단의 **관리 안정성** 또한 나타내기 위해 제작한 웹앱 프로젝트

React, Javascript, React-Query, Docker, Nginx, Github-Actions, AWS EC2, Artillery



트러블 슈팅 | 1. 방사선 페이지 리렌더링 성능 개선 [트러블 슈팅 회고](#) [링크 2](#)

문제 상황

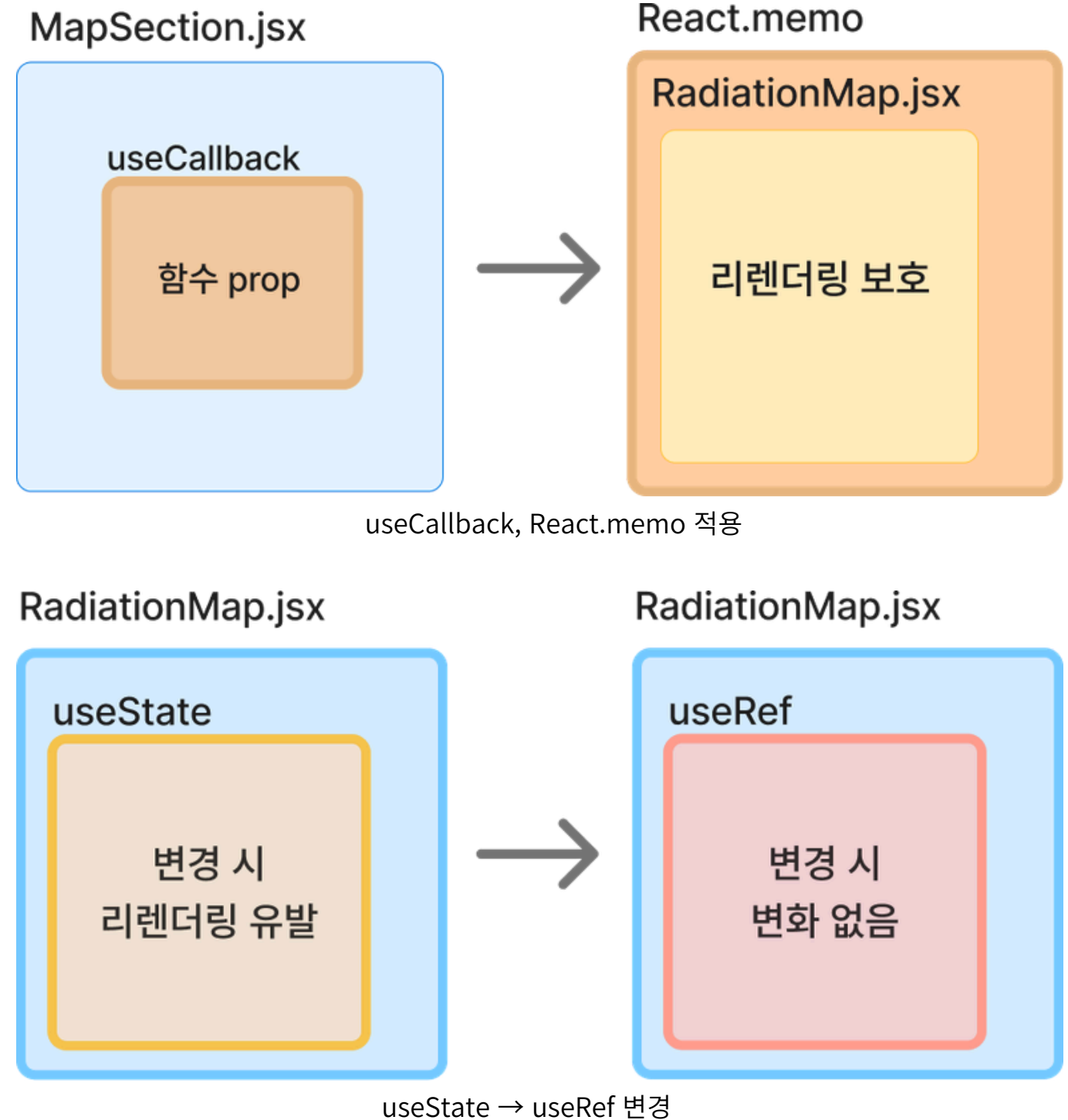
방사선량 시각화 페이지의 렌더링 시간이 다소 지연되던 이슈가 있었습니다. 원인을 찾기 위해 React Devtools를 사용했고, RadiationMap 컴포넌트가 총 5회 렌더링 되는 것을 발견할 수 있었습니다. 이는 예상했던 것보다 많은 렌더링 횟수였으며, 렌더링 원인은 props 및 state 변경이었습니다.

해결 과정

먼저 해당 컴포넌트는 함수 타입의 props를 받았는데, 리액트에서는 얇은 참조로 props를 가져오기 때문에 매 렌더링마다 다른 props로 인식되어 리렌더링이 발생했습니다. 이를 해결하기 위해 React.memo와 useCallback을 사용해 부모 컴포넌트의 리렌더링에도 함수 참조값이 변경되지 않도록 했고, 이에 따라 자식 컴포넌트는 변경된 props로 인식하지 않았습니다. 둘째로, 컴포넌트의 state 남용으로 발생한 리렌더링이었습니다. 본래 state는 리렌더링을 유발해야 하는 상황에서만 사용해야 하지만, 이전 코드에서는 값을 저장하여 useEffect 실행을 유발하는 용도로 사용했습니다. 이는 어떤 상태가 변경되어 함수가 실행되는지 알 수 있어 가독성은 증가하겠지만 불필요한 렌더링을 유발하여 웹 성능을 저하시키고, 렌더링 비용이 큰 해당 컴포넌트에서는 나쁜 선택이었습니다. 값을 저장하지만 렌더링을 유발하지 않는 useRef로 변경하고 두 개의 useEffect를 통합하는 방식으로 수정하여 리렌더링 횟수를 감소시켰습니다.

결과 및 배운 점

최종적으로 렌더링 시간은 60% 개선, 렌더링 횟수는 3회 감소시킬 수 있었습니다. 카카오맵이 포함되어 리렌더링 비용이 크기 때문에 불필요한 리렌더링을 최소화해야 했습니다. 해당 이슈를 통해, 렌더링 성능을 위해 기술적인 최적화뿐만 아니라 올바른 상태(state, props 등)를 사용하고 컴포넌트를 설계하는 것도 중요하다는 것을 깨닫게 되었습니다.



트러블 슈팅 | 2. 이미지 로딩 성능 이슈 해결 [트러블 슈팅 회고](#)

문제 상황

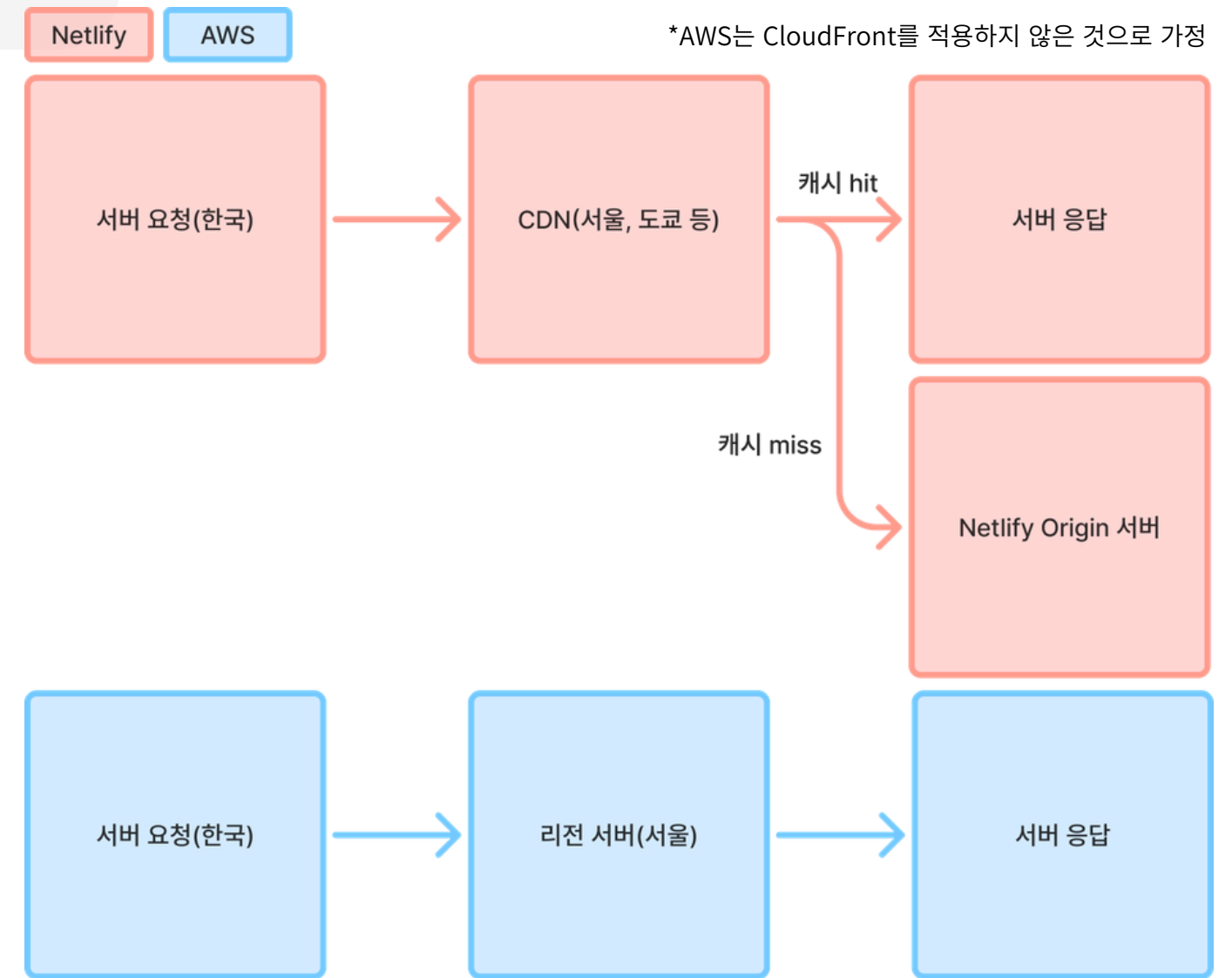
프론트엔드 netlify 배포 이후, 이미지 로딩 성능 저하 이슈를 발견했습니다. lighthouse를 통해 Core Web Vitals를 확인해 보니, CLS, TTFB, LCP가 낮은 성능으로 나타났으며 전체 점수는 평균 60점 미만이었습니다. 이 지표를 토대로 다음과 같은 단계를 거쳐 개선했습니다.

해결 과정

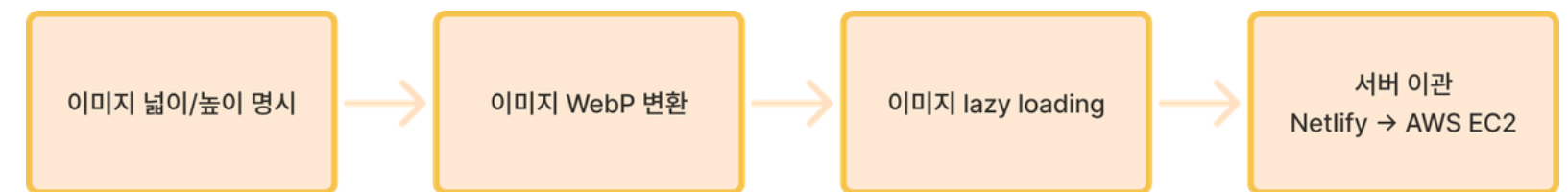
가장 먼저 Layout Shift 방지를 위해 이미지의 width와 height를 명시하여 브라우저 사전 공간을 확보했습니다. 또한, 이미지의 형식을 webp로 변환하여 손실/무손실 압축을 적용했고, 이미지 lazy 로딩 방식을 적용했습니다. CLS와 FCP는 다소 로딩 성능이 체감될 만큼 개선되지 않았습니다. 이에 따라 로딩 성능 저하의 원인을 **서버 성능 저하**로 판단했습니다. Netlify는 CDN을 사용하여 해외 이용자에게도 비교적 빠른 응답을 지원하지만 네트워크 상황에 따라 대부분 도쿄에서 캐싱된 응답을 반환하고, 캐시 미스 시에는 해외 오리진 서버에 요청됩니다. 서버 요청에 있어 거리는 곧 성능과 직결되기 때문에 서울 리전을 지원하는 AWS보다 평균 응답 속도가 느릴 것이라 판단했고, 이러한 근거를 바탕으로 AWS S3 이전을 통해 서버 성능 및 로딩 성능을 개선할 수 있었습니다.

결과 및 배운 점

결과적으로 lighthouse 성능 점수를 평균 60점에서 90점으로 개선할 수 있었고, 평균 LCP(3.2s → 1.8s), TTFB(950ms → 250ms), CLS(0.3 → 0.08)가 개선되었습니다. 사실상 이번 이슈는 TTFB로 인한 이슈였고, 이를 개선하니 다른 지표들도 간접적으로 개선되었습니다. 각 지표 분석을 통해 성능 저하의 원인을 쉽게



Netlify, AWS의 서버 요청 과정



이미지 로딩 성능 이슈 해결 프로세스